Unit 3

Arrays

Unit 3 Syllabus

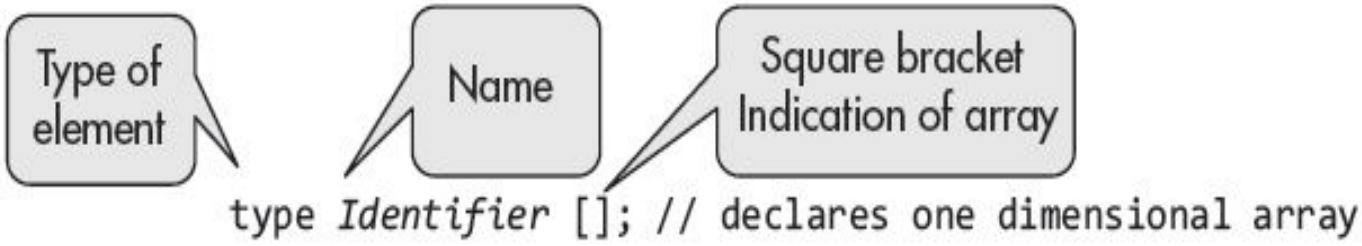
- **Arrays:** Introduction, Declaration and Initialization of Arrays, Storage of Array in Computer Memory, Accessing Elements of Arrays, Operations on Array Elements, Assigning Array to Another Array, Dynamic Change of Array Size, Sorting of Arrays, Search for Values in Arrays, Class Arrays, Two-dimensional Arrays, Arrays of Varying Lengths, Three-dimensional Arrays, Arrays as Vectors.
- **Inheritance:** Introduction, Process of Inheritance, Types of Inheritances, Universal Super Class-Object Class, Inhibiting Inheritance of Class Using Final, Access Control and Inheritance, Multilevel Inheritance, Application of Keyword Super, Constructor Method and Inheritance, Method Overriding, Dynamic Method Dispatch, Abstract Classes.
- **Interfaces:** Introduction, Declaration of Interface, Implementation of Interface, Multiple Interfaces, Nested Interfaces, Inheritance of Interfaces, Default Methods in Interfaces, Static Methods in Interface.

Introduction

- When we need to process a large number of values of the same type, an array can store them efficiently.
- All the elements of an array are represented using one name in computer memory.
- The data type of the elements stored in an array is called the **base type** of the array.
- In Object-Oriented Programming (OOP), an array is a sequence of objects of the same data type.
- An array can store objects of a particular class, but it cannot directly contain elements of different data types.

Declaration of Arrays

- Declaration starts with the type, followed by an identifier and a square bracket, and ends with a semicolon.



Examples:

int numbers []; // an array of whole numbers
char name []; // A name is an array of characters
float priceList []; // An array of floating point numbers.

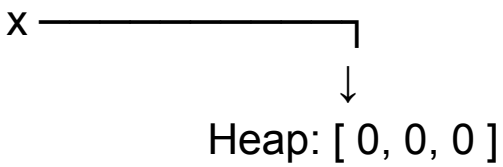
int numbers[]=new int[4]; //allocates memory for 4 int values
char name []=new char[6]; //allocates memory for 6 characters

- int x[]; // declaration only
  - This statement:
    - Declares an array reference variable x of type int[], but it does not create an actual array.
    - No memory is allocated for an array on the heap, and x is not yet pointing to an array object. Instead, it holds a **null** reference by default
  -

int x[]; // Declaration only

x->null;

- **int x[] = new int[3];**
- This statement:
  - **Declares** and **initializes** the array reference x.
    - Allocates memory on the heap for an array of 3 integers.
    - Initializes each element in the array to the default value of 0 for int.
  - **Memory**:
    - Stack: A reference variable x is created on the stack, and it points to the array object in the heap.
    - Heap: Memory for an array of 3 integers is allocated, with each element initialized to 0
    - **Stack:**



Initialization of Arrays

- An array may be initialized by mentioning the values in braces and separated by commas.

- For example, the array pencils may be initialized as below:

```
int pencils [] = {4, 6, 8, 3};  //Array initializer shorthand
```

if you want to use new operator

```
int pencils [] = new int[]{4, 6, 8, 3};
```

- Shorthand syntax - int[] numbers = {1, 2, 3};
- This is valid **only when declaring and initializing the array at the same time**.
- But this is **not valid**:
  - numbers = {4, 5, 6}; // ERROR
- Why? Because {4, 5, 6} by itself is an **array initializer**, and Java does not allow this syntax for a later assignment.

- Using new int[]
  - int[] numbers = new int[]{1, 2, 3};
  - This creates a new array object.
  - It can also be used later for **assignment/ reassignment**:
  - numbers = new int[]{4, 5, 6};
  - Now numbers refers to the new array:

    numbers → [4, 5, 6]
  - {...} → shorthand, mainly used during declaration
  - new int[]{...} → creates a new array object and can also be used for reassignment.

Aspect	**int pencils[] = {4, 6, 8, 3};**	**int pencils[] = new int[]{4, 6, 8, 3};**
Usage	Only for declaration and assignment	For declaration, assignment and reassignment
Syntax	Shorter	Explicit, requires new int[]
Flexibility	Cannot be used for reassigning an array	Can be used for both declaration and reassignment

Alternative Array Declaration Syntax

type [] *identifier*; // declares one dimensional array

type [][] *identifier*; // declares two dimensional array

E.g.

int [] numbers

int [] numbers, items, guns; // declares three arrays

int[] number[]; is equivalent to

int number[][]; or int[][] numbers;

int[][][] number;

int[][] number[];

int[] number[][];

int number[][][];

Storage of Array in Computer Memory

- The operator *new* allocates memory for storing the array elements.
- For example, with the following declaration

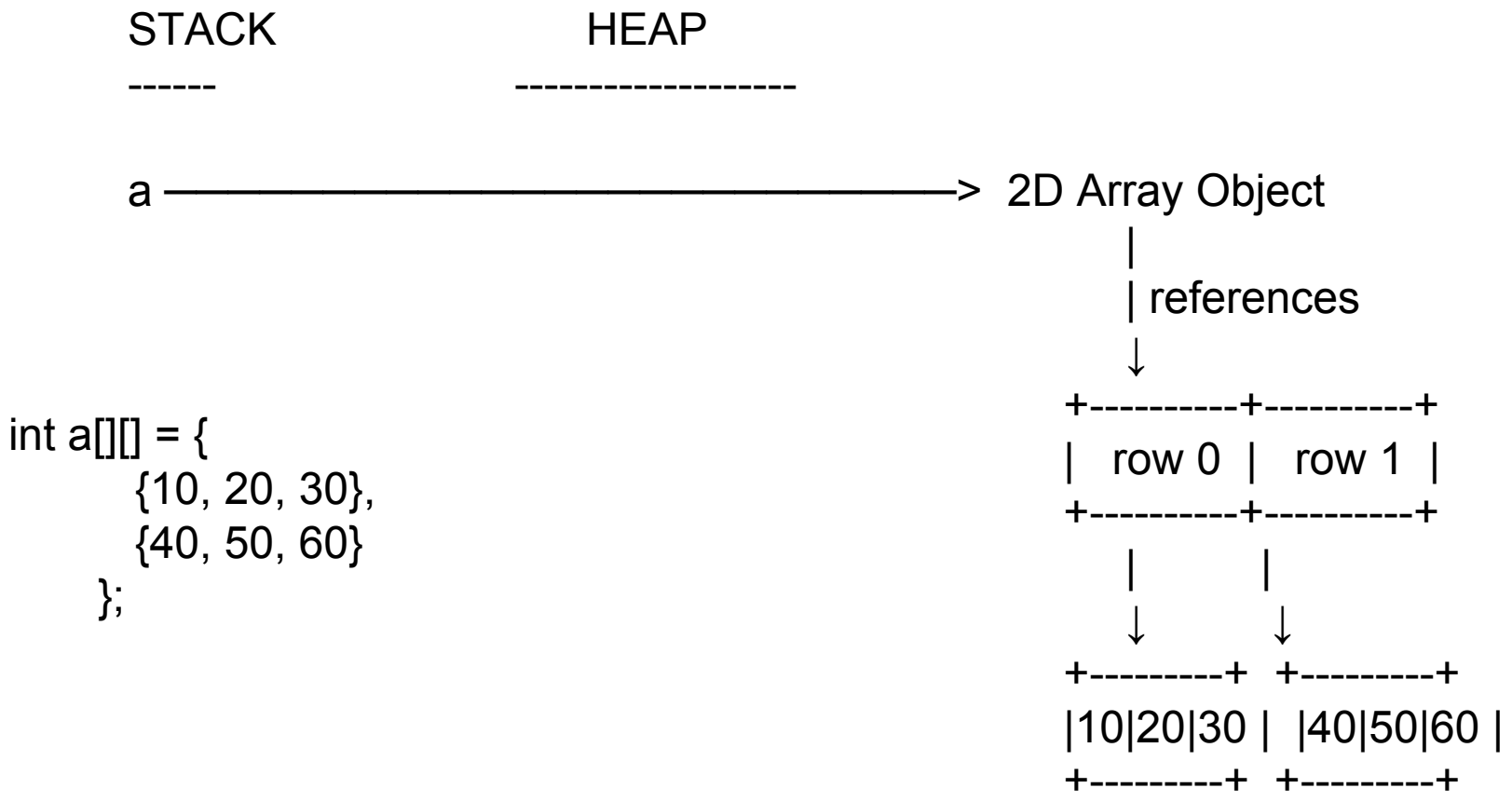
  int[] numbers = new int[4];
- The declaration and initialization may as well be combined as:

  int numbers [] = {20,10,30,50};
- A two-dimensional array may be declared and initialized as,

  int [][] array2d = new int [][] {{1, 2, 3}, {4, 5, 6}};

  or as

  int [][] array2D = {{1, 2, 3}, {4, 5, 6}};


STACK
HEAP
a
2D Array Object
references
row 0
row 1
int a[][] = {
{10, 20, 30},
{40, 50, 60}
};
|10|20|30 |
|40|50|60 |

Accessing Elements of Arrays

- An array stores multiple elements, and **each element can be accessed using its index number**.
  - The **index** indicates the **position of an element** in the array.
  - In Java, array indexing **starts from 0**.
  - Therefore, the **first element** is at index **0**, the second at index **1**, and so on.
  - The last element is always at index **length - 1**.

```
Array:  [80] [75] [90] [85]
Index:    0    1    2    3
```

So, **array[index]** is used to access a particular element of an array.

Use of *for–each* Loop

- the *for–each* loop may be used to access each element of the array.

```
for (int x: numbers)
        System.out.println(x);
```

- For a two-dimensional array the nested *for–each* loops are used.

```
int pArray [][]= {{1,2,3},{4,5,7}};
for(int [] y : pArray)
{
    for(int x : y)
        System.out.print(x + " ");
 }
```

ArrayEx1.java

Determination of Array Size

- The size or length of an array may be determined using the following code:

```
int arraySize = array_identifier.length;
```

ArrayEx2.java

Array of Strings

```
class ArrayEx3 {
    public static void main(String[] args) {
        // Declare and initialize an array of strings
        String[] fruits = {"Apple", "Banana", "Cherry", "Date"};

        // Access and print each element of the array
        for (int i = 0; i < fruits.length; i++) {
            System.out.println(fruits[i]);
        }
    }
}
```

ArrayEx3.java

Operations on Array Elements

Arithmetic operations on Arrays:

```
class ArrayEx4 {
    public static void main(String[] args) {
        int[] array1 = {1, 2, 3};
        int[] array2 = {4, 5, 6};
        int[] sumArray = new int[array1.length];

        for (int i = 0; i < array1.length; i++) {
            sumArray[i] = array1[i] + array2[i];
        }

        System.out.println("Sum of arrays:");
        for (int num : sumArray) {
            System.out.print(num + " ");
        }
    }
}
```

```
diffArray[i] = array1[i] - array2[i];
mullArray[i] = array1[i] * array2[i];
divArray[i] = array1[i] /array2[i];
newArray[i] = array1[i] +3;
```

ArrayEx4.java

Arrays as parameters to methods

- When an array is passed to a method, it is passed by reference, which means the method receives a reference to the original array, not a copy.
- Any changes made to the array inside the method will affect the original array.

ArrayEx5.java

Assigning Array to Another Array

- In Java, an array may be assigned to another array of same data type.
- **Assigning one array to another** means that both arrays will refer to the same memory location.
- This means that the new array will not be a copy of the original; instead, it will point to the same array object in memory.
  - Any changes made to one array will reflect in the other.

ArrayEx6.java

Dynamic change of Array size

- The number of elements (size) of the array may change during the execution of the program.
- This feature is unlike C and C++.
- In Java, you may change the number of elements by dynamically retaining the array name.
- In this process, the old array is destroyed along with the values of elements.

ArrayEx7.java

Sorting of Arrays

- Sorting of arrays is needed in many applications of arrays.
- The arrays may be sorted in ascending or descending order.
- Several methods are used for sorting the arrays that include the following:

  1. Bubble sort
  2. Selection sort
  3. Sorting by insertion method
  4. Quicksort

Bubble Sort

- This method is the simplest but the most inefficient.
- It can be usefully employed for short arrays. In the process, if the sorting is done in ascending order, the first element is compared with the second element. If the value of the second is smaller than the first, the elements are interchanged, that is, the second is made first and first is made second.
- This method is the simplest but the most inefficient.
- It can be used for short arrays.
- In the process, if the sorting is done in ascending order, the first element is compared with the second element.
- If the value of the second is smaller than the first, the elements are interchanged.
- The process is repeated till the last element.

Bubble Sort

- This process places the largest value as the last element.
- The process is again repeated for the next largest value from the remaining elements until the last element of the array is reached.
- Thus, the process is repeated ($n-1$) times to completely sort the array.

Bubble Sort

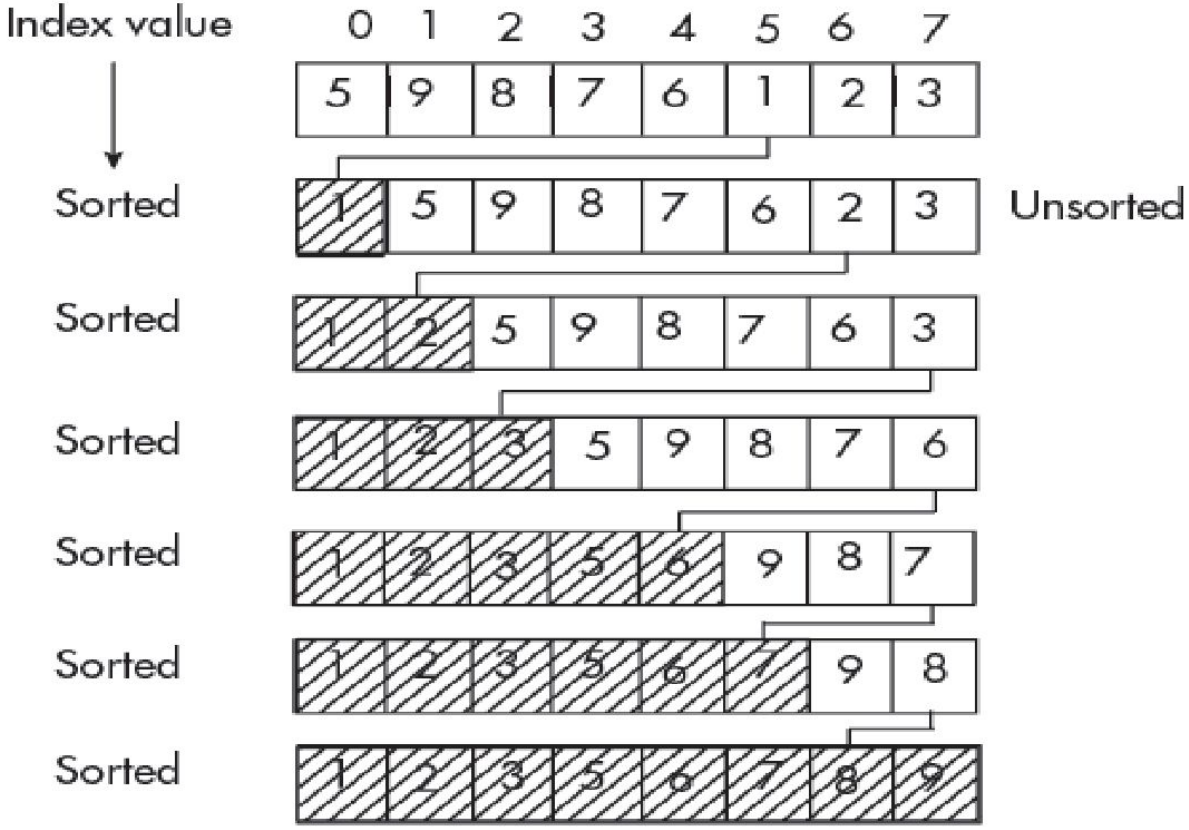


Fig. 7.6 Bubble sort method

Program Example

Program 7.9: Illustration of array sorting by Bubble sort method

```
1    public class  BubbleSort     {
2    public static void main(String[] args) {
3
4    double darray[] = new double[]{7.7,1.3,9.9,1.1, 6.6, 2.2,
     3.3, 8.8, 4.4, 5.5};
5    bubbleSort (darray);
6      for (int k =0; k <10; k++)
7      System.out.print(darray [k] + "    ");
8      System.out.println();
9        }
10
11
12   public static void bubbleSort (double[] x)
13   {int length = x.length;
14      double y;
15      for(int i = 0; i<length; i++)
16     {for(int j = length-1; j >=(i+1); j--)
17      {if (x[j]  < x[j-1])
18      {    y = x[j];
19       x[j] = x[j-1];
20       x[j-1]=y;
21       }}}
22    }
23   }
```

Selection Sort

- Let us take an array with elements 9, 7, 4, 3, 6, and 8.
- If the array is being sorted in ascending order, pick the element with the lowest value, that is 3, and place at the first place as shown in the second line of Figure.
- From the remaining elements that are 9, 7, 4, 6, 8, again pick the lowest value, that is, 4 and place it next to 3, as shown in the third line of the figure.
- Then, from the remaining elements with values 9, 7, 6, 8, again pick the lowest value, that is, 6 and place it next to 4.
- From the remaining three values, 9, 7, 8, again pick the lowest value, that is, 7 and place it next to 6 on its right side.
- Then, from the remaining two values 9 and 8, pick the lowest, that is, 8 and place it next to 7.
- The highest value goes to the last place. The array is sorted with the lowest value at the first place and the largest value at the end.
- The process is better than the bubble sort but still not the most efficient.

Selection Sort

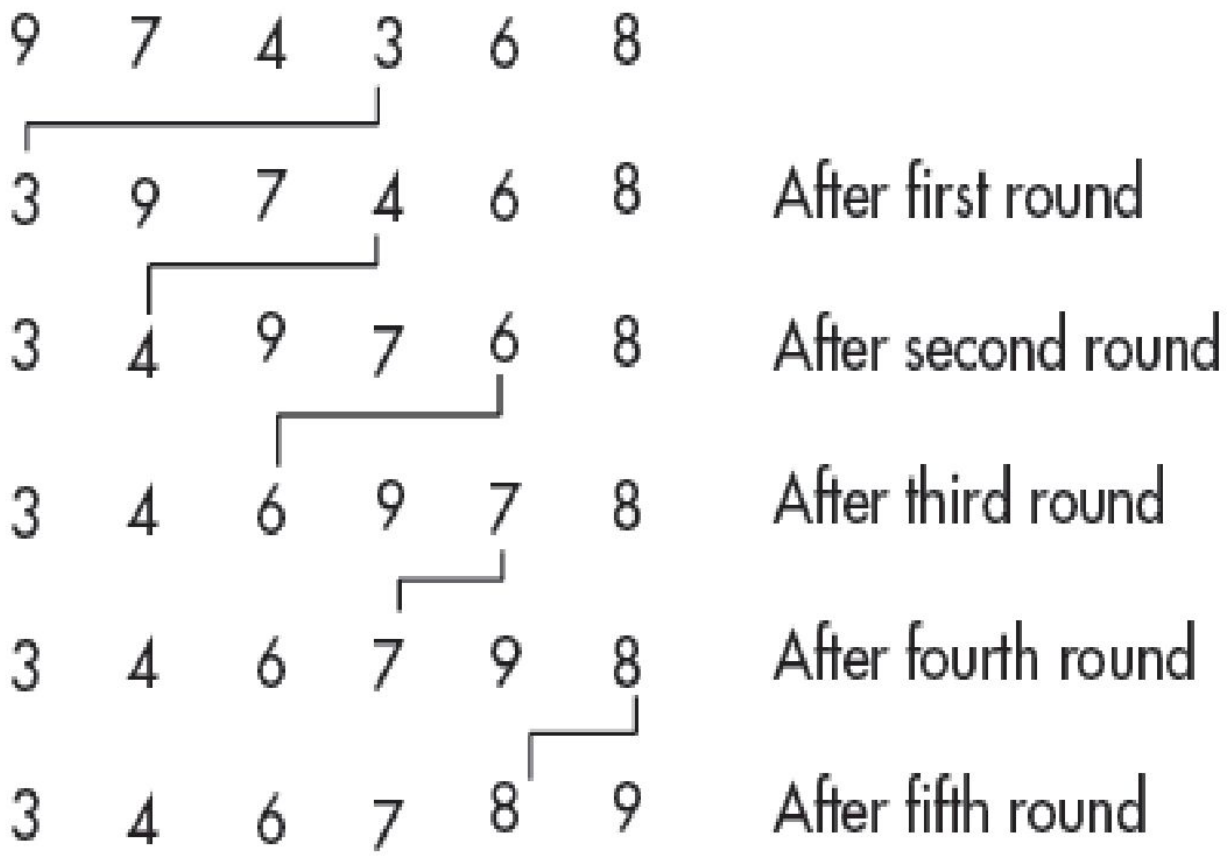


Fig. 7.7 Selection sort

Sorting by Insertion Method

- Sorting algorithm builds a final sorted array one item at a time.
- In this method, the value at any index is compared to all the prior elements.
- The input data is inserted into the correct position in the sorted list and the process is repeated until no input element remains.

Sorting by Insertion Method

9	7	4	3	6	8	
7	9	4	3	6	8	After first round
4	7	9	3	6	8	After second round
3	4	7	9	6	8	After third round
3	4	6	7	9	8	After fourth round
3	4	6	7	8	9	After fifth round

Fig. 7.8 Insertion sort

Program Example

Program 7.11: Illustration of insertion method of sorting

```
 1       public class SortByInsertion {
 2      public static void main(String[] args) {
 3
 4      int intarray [] = new int [] {100,30,20,90,80,60,50,40};
 5          sort (intarray);
 6          for (int k=0 ; k < intarray.length; k++)
 7          {  System.out.print(intarray[k]+"    ");
 8            System.out.println();
 9          }
10          }    // end of main method
11
12      // below is definition of insertion sort method
13       public static void  sort(int[] array){
14        int length = array.length;
15        int key = 0;
16
17        int i = 0;
18        for(int j = 1;j<length;j++){
19        key = array[j];
20           i = j-1;
21        while(i>=0 && array[i]>key)
22          {  array[i+1] = array[i];
23           i = i-1;
24           array[i+1]=key;
25          }
26        for (int m=0; m<length; m++)
        //display of sort process
27        System.out.print(array[m] + "    ");
28        System.out.println ( );
29        }}
30       }
```

Search for Values in Arrays

- Searching an array for a value is often needed.
- Let us consider the example of searching for your name among the reserved seats in a rail reservation chart, etc.
- Two methods are employed: linear search and binary search for sorted arrays.

Linear Search

- The method may be applied to any array.
- The key value is compared to the value of the elements of the array successively.
- If a match is found, it is noted, and the program ends there.
- Otherwise, the complete array is searched, and if no match is found, it is reported that the value is not there in the array.

Linear Search

Value to be searched = 45

Index value	0	1	2	3	4	5	6	7
	15	23	81	77	25	45	54	12

Compare with 15: 15! = 45
Compare with 23: 23! = 45
Compare with 81: 81! = 45
Compare with 77: 77! = 45
Compare with 25: 25! = 45
Compare with 45: 45 == 45
The value is found at index value 5.

Fig. 7.9 Illustration of linear search

Program Example

Program 7.12: Illustration of linear search for value array

```
1   import java.util.Scanner;
2   public class ArraySearch {
3     public static void main(String[] args) {
4       boolean b = false ;
5       int [] array = {67, 78, 85, 44, 34, 25, 65, 36};
6       Scanner scanin = new Scanner(System.in);
7       System.out.print("Enter the number you want to search: ");
8
9       int key = scanin.nextInt();
10        //Comparison of key with array elements
11      for (int i = 0; i<array.length; i++)
12        if(array[i] == key)
13      {System.out.println("Your number is at index =" + i);
14          b = true;
15      }
16    if(b!=true)
17  System.out.println("Your number is not in the array.");
18  }
19  }
```

Binary Search

- It is a very efficient method of search but it is applicable only to the sorted arrays.
- The beginning, end, and the midpoint of the array are defined first.
- The key value is compared with the value at midpoint.
- If it does not match, then it is checked in which half of the array the key value lies by checking whether the key value is more or less than the value at midpoint.

Binary Search

- The array is truncated to the half in which the key value lies.
- This process is repeated on that half, that is, it is again divided into two halves where the value is compared with the midpoint;
- if match is not found, it is determined in which half of the truncated array the value lies. The array is again truncated to the half in which the key value lies. The process is repeated till the key value match is found. The process is very useful for searching large sorted arrays.

Binary Search

- The array is truncated to the half in which the key value lies.
- This process is repeated on that half, that is, it is again divided into two halves where the value is compared with the midpoint;
- if match is not found, it is determined in which half of the truncated array the value lies.

Binary Search

The process is very useful for searching large sorted arrays

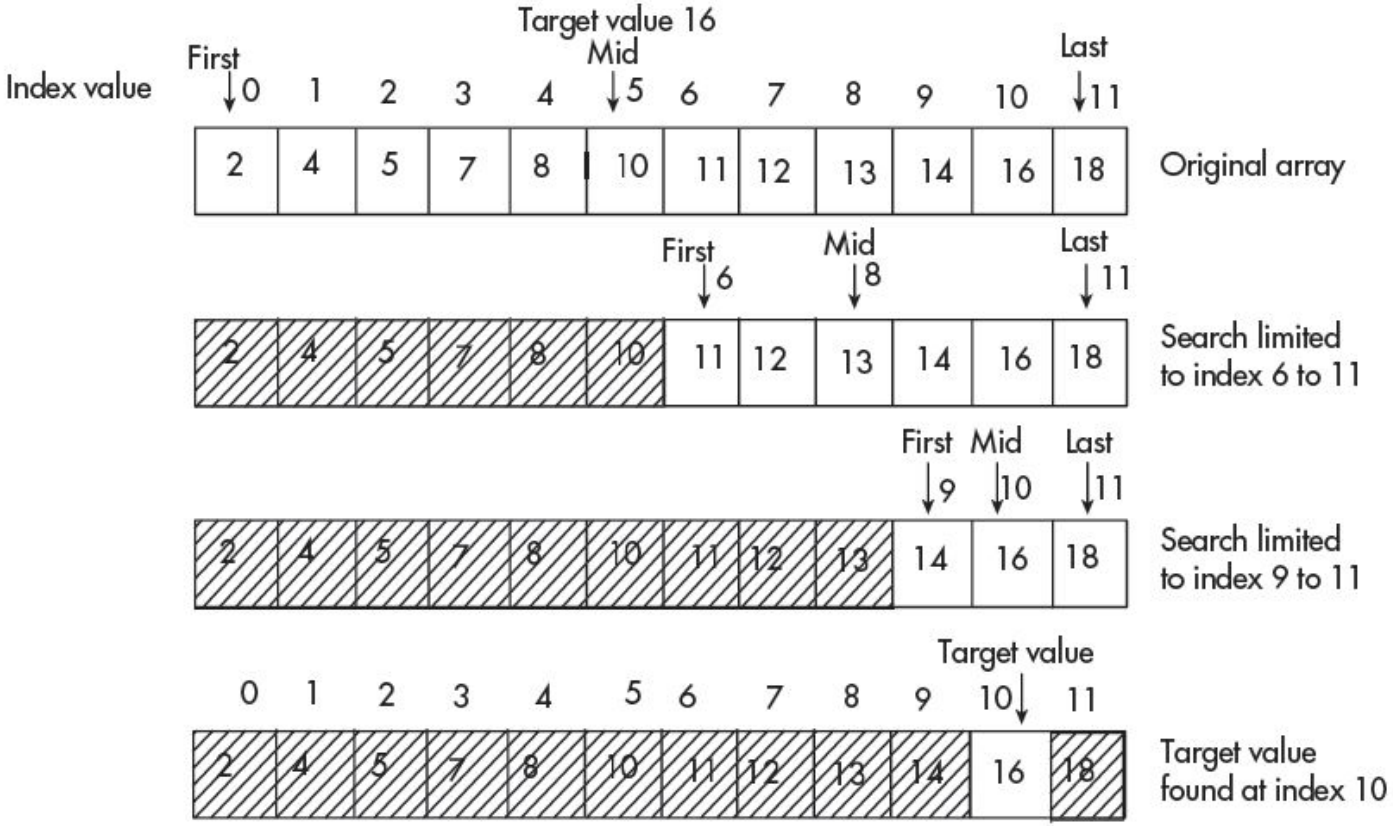


Fig. 7.10 Illustration of binary search

Program Example

Program 7.13: Illustration of binary search of sorted arrays

```
1     import java.util.Scanner;
2     public class BinarySearch {
3      public static void main(String[] args) {
4
5     int[] array ={15, 20, 43, 45, 76, 80, 86, 88, 90, 94, 96, 98};
6
7      int length = array.length;
8      Scanner scanin = new Scanner(System.in);
9      System.out.print("Enter the number you want to search: ");
10     int key = scanin.nextInt();
11     int beginning, end, mid;
12     beginning = 0;
13     end = length -1;
14     while (beginning <= end)
15        {
16        if (beginning == end && array[end] != key)
17     { System.out.println("Your number is not in the array");
18      break;
19      }
20      mid = (beginning + end)/2;
21    if (array[mid]== key)
22    {System.out.println(" The key value is in the array at index "+ mid);
23
24       break;}
25       else if( array[mid]< key)
26       beginning = mid +1;
27     else
28       end = mid -1;
29        }
30     }
31    }
```

Two-dimensional Arrays

- An array can contain other arrays as its elements.
- If each element is a one-dimensional array, it forms a two-dimensional array.
- Therefore, a 2D array in Java is actually an array of arrays.
- Each row can have the same or different number of elements.
- A matrix is a common example of a 2D array.
- A list of telephone numbers can also be represented using a 2D array.

```
int[][] a = {
    {10, 20, 30},
    {40, 50},
    {60, 70, 80, 90}
};
```

Two-dimensional Arrays

- A two-dimensional array may be defined as:

  int telNumber [][] = new int [5][10];

  This creates a **2D array with 5 rows and 10 columns**.

A two-dimensional array may as well be defined and initialized as

  int arrayB [ ][ ] = {{11, 12, 13 }, {7, 6, 4}};

- The two-dimensional array may as well be declared as

  int arrayC [][] = new int[2][3]{{1,2,3}, {4.5.6}}

Arrays of Varying Lengths

- A two-dimensional array is treated as an array whose elements are one-dimensional arrays, which may have different sizes.
- A two-dimensional array may be declared as

```
int a2D [][] = new int [3 ][];
```

- The arrays may as well be declared as

```
int array [][] = {
        {5, 7, 8 },{10, 11 }, {4, 3, 2, 7,5 }
 };
```

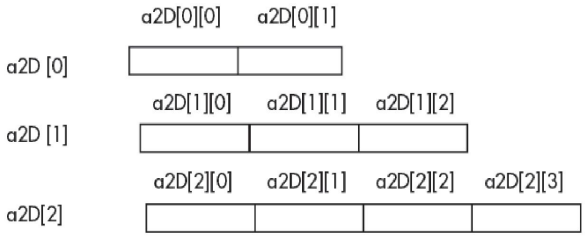


Fig. 7.11 Two-dimensional arrays of varying lengths

Program Example

Program 7.20: Illustration of arrays of varying lengths

```
1    public class Array2D2 {
2     public static void main(String[] args)
3    {
4    int [][] array = new int [3][];
5       array[0]  = new int[]{ 6, 7};
6       array[1] = new int []{10,11,12,14};
7       array[2] = new int [] { 2,3,4};
8
9    for (int i =0; i<3;i++)
10       {for ( int j = 0; j<array[i].length; j++)
11       System.out.print(array[i][j] + "   ");
12       System.out.println();}
13   }
14   }
```

Three-dimensional Arrays

- When an array holds two-dimensional arrays as its elements, the array is a three-dimensional array.
- Each basic element of such an array needs three index values for its reference.
- A three-dimensional array may be declared as

```
int tDArray [][][]; //Declaration
double d3Array [][][]; //Declaration
```

Class Arrays

ArrayEx8

- The package java.util defines the class **Arrays** with static methods.
-

Method	Purpose
public static void sort(int[] a)	Sorts the elements of an array
public static void fill(int[] a, int val)	Fills an array with a specified value
public static int[] copyOf(int[] original, int newLength)	Copies an array, possibly with a different length
public static boolean equals(int[] a, int[] a2)	Compares two arrays for equality
public static int binarySearch(int[] a, int key)	Searches for an element in a sorted array
public static String toString(int[] a)	Converts an array to a string representation
public static String deepToString(Object[] a)	Converts a multidimensional/nested array to a string
public static int hashCode(int[] a)	hash code for the contents of the integer array.
public static <T> List<T> asList(T... a)	Converts an array into a List

Vectors

- **Vector** is a collection class used to store multiple elements.
- It is similar to an **array**, but unlike an array, a Vector can **grow or shrink dynamically**.
- Therefore, Vector can be used to implement a **dynamic array**.
- Vector is available in the java.util package.

Vector Constructors

- **Vector()** - Creates a Vector with a **default initial capacity of 10**.
  Vector<Integer> v = new Vector<>();
  - Initial Capacity = 10 Size 0
- **Vector(int capacity) -** Creates a Vector with the specified **initial capacity**.
  Vector<Integer> v = new Vector<>(5);
  - Capacity is 5 - Size 0

Remember:
Size = number of elements currently stored.
Capacity = number of elements it can store before reallocation.

- **Vector(int capacity, int incr)**
  - capacity → initial capacity
  - incr → capacity increase when more space is required
  - Vector<Integer> v = new Vector<>(5, 3);
  - Capacity 5, size 3

Vectors

1. **Dynamic memory allocation**
   A Vector can automatically increase its capacity when more elements are added.
2. **Resizable**
   The size of a Vector can **grow or shrink dynamically** according to the requirement.
3. **Stores a dynamic list of objects**
   Unlike a fixed-size array, a Vector can maintain a **variable number of elements**.
4. **Easy insertion and deletion**
   Elements can be **added or removed** using methods such as add() and remove().

Methods of Vector class

Method	Simple meaning	Example
add(Object element)	Inserts element at end	v.add(50);
add(int index, Object element)	Inserts element at a particular index	v.add(1, 50);
addElement(Object obj)	Adds element at the end	v.addElement(40);
clear()	Removes all elements	v.clear();
capacity()	Returns current capacity	v.capacity();
copyInto(Object[] array)	Copies Vector elements into an array	v.copyInto(a);
firstElement()	Returns first element	v.firstElement();
lastElement()	Returns last element	v.lastElement();
elements()	Returns an Enumeration for traversing elements	v.elements();
get(int index)	Returns element at given index	v.get(1);
remove(int index)	Removes element at given index	v.remove(1);
size()	Returns number of elements	v.size();